

Lecture# 11

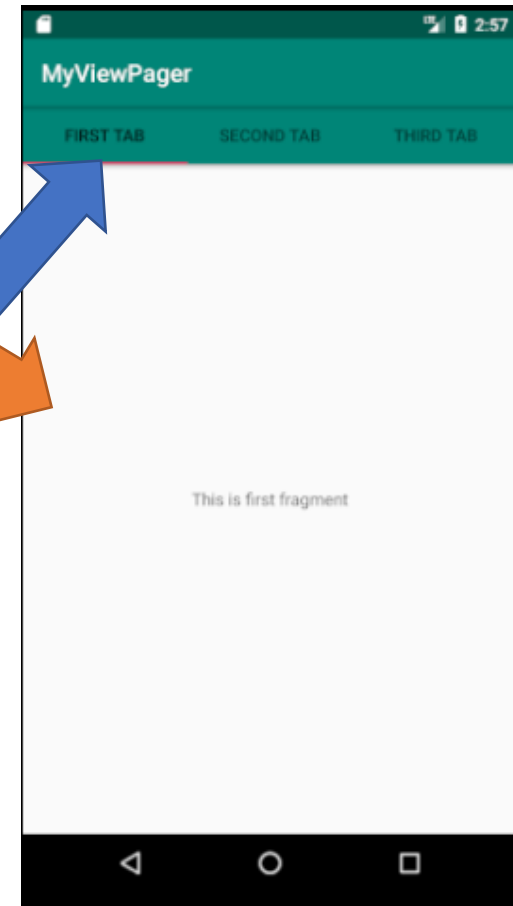
View Pager

By

Dr. Muhammad Bilal

View Pager

- ViewPager is a class/layout manager that allows the user to swipe/flip left and right through pages of application.
- Adapter
 - The adapter is used to generate the fragment or pages on the ViewPager
- Tabs
 - To make the ViewPager more user friendly, the tabs can be added on top or bottom of the view pager



Fragments in ViewPager

- The fragment are used in ViewPager to display the view in android application
- For each page in ViewPager, you have to create and design the fragment for that view to display

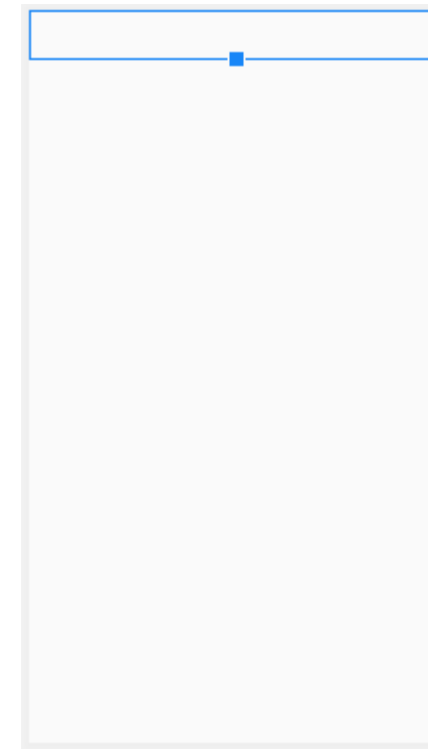
ViewPager and Tablayout in Layout

- To design a ViewPager in your android application, first you have to design a ViewPager in your main activity layout
- To add a tab layout in the view pager, first you have to add the tab using “TabLayout” in the main activity XML layout

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <com.google.android.material.tabs.TabLayout
        android:id="@+id/tabs"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />

    <androidx.viewpager.widget.ViewPager
        android:id="@+id/view_pager"
        android:layout_width="match_parent"
        android:layout_height="match_parent" />
</LinearLayout>
```



Fragment for ViewPager

- Next the fragment can be design which will be displayed on ViewPager,
- Consider three simple fragments and their layout



Fragment for ViewPager (Cont.)

```
public class FirstFragment extends Fragment {
    @Nullable
    @Override
    public View onCreateView(@NonNull LayoutInflater inflater, @Nullable ViewGroup container,
        @Nullable Bundle savedInstanceState) {
        return inflater.inflate(R.layout.first_fragment, container, false);
    }
}

public class SecondFragment extends Fragment {
    @Nullable
    @Override
    public View onCreateView(@NonNull LayoutInflater inflater, @Nullable ViewGroup container,
        @Nullable Bundle savedInstanceState) {
        return inflater.inflate(R.layout.second_fragment, container, false);
    }
}

public class ThirdFragment extends Fragment {
    @Nullable
    @Override
    public View onCreateView(@NonNull LayoutInflater inflater, @Nullable ViewGroup container,
        @Nullable Bundle savedInstanceState) {
        return inflater.inflate(R.layout.third_fragment, container, false);
    }
}
```

Creating ViewPager Adapter

- Before adding ViewPager in the main activity, the ViewPager's page adapter must be implemented using "FragmentPagerAdapter"

```
public class MyPagerAdapter extends FragmentPagerAdapter {  
    public MyPagerAdapter(FragmentManager fm) {  
        super(fm);  
    }  
  
    @Override  
    public Fragment getItem(int position) {  
        return null;  
    }  
  
    @Override  
    public int getCount() {  
        return 0;  
    }  
}
```

```
    @Nullable  
    @Override  
    public CharSequence getPageTitle(int position) {  
  
        return null;  
    }  
}
```

FragmentPagerAdapter

- The FragmentPagerAdapter has following methods that's must be implemented in the adapter
 - Constructor
 - This is the constructor of the adapter class which get fragment manager as parameter

```
MyPagerAdapter(FragmentManager fm)
```

- getItem
 - It returns the object of the fragment for specified position

```
Fragment getItem(int position)
```

- getItemCount
 - It will return the total number of the pages in the ViewPager

```
int getCount()
```


FragmentPagerAdapter

- getTitle()
 - This method in view pager adapter function to return title for each tab in the view pager adapter class

```
@Nullable
@Override
public CharSequence getTitle(int
position) {

    return null;
}
```

MyPagerAdapter.java

```
public class MyPagerAdapter extends FragmentPagerAdapter {
    public MyPagerAdapter(FragmentManager fm) {
        super(fm);
    }

    @Override
    public Fragment getItem(int position) {
        switch(position){
            case 0:
                return new FirstFragment();
            case 1:
                return new SecondFragment();
            case 2:
                return new ThirdFragment();
        }
        return null;
    }

    @Override
    public int getCount() {
        return 3; // since we have three fragments
    }
}
```

```
    @Nullable
    @Override
    public CharSequence getPageTitle(int position) {

        switch(position){
            case 0:
                return "First Tab";
            case 1:
                return "Second Tab";
            case 2:
                return "Third Tab";
        }
        return null;
    }
}
```

Add ViewPager in MainActivity

- In the main activity, the object of page adapter class is initialized by passing fragment manager as parameter

```
MyPagerAdapter pageAdapter = new MyPagerAdapter(getSupportFragmentManager());
```

- Next is to initialize the view pager UI component

```
ViewPager viewPager = (ViewPager)findViewById(R.id.pager);
```

- Initialize the tab UI component and set the view pager object with the tab method

```
TabLayout tab = findViewById(R.id.tabs);  
tab.setupWithViewPager(viewPager);
```

- Now set the adapter for ViewPager, which is initialized above

```
viewPager.setAdapter(pageAdapter);
```

MainActivity.java

```
public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        MyPagerAdapter pageAdapter = new MyPagerAdapter(getSupportFragmentManager());
        ViewPager viewPager = findViewById(R.id.view_pager);

        TabLayout tab = findViewById(R.id.tabs);
        tab.setupWithViewPager(viewPager);

        viewPager.setAdapter(pageAdapter);
    }
}
```

